Individual Testing

1. Assignment & Accountability

Contributor / Member ID	Rola Refaat
Type of vulnerability	Web Application Security – Cross-Site Request Forgery (CSRF)
Week / reporting period	Week 2 – Exploitation Testing
Target / environment	OWASP Juice Shop (http://127.0.0.1:3000 / localhost:3000)

2. Objective, Scope & Preconditions

Objective: Check if the /profile endpoint on the local Juice Shop app is vulnerable to CSRF, i.e. can an attacker change a logged-in user's username without them doing it themselves.

In-Scope Asset: Local OWASP Juice Shop at http://127.0.0.1:3000 (localhost:3000).

Account / Role Used: A normal test "customer" account (ddd@gmail.com, egs@gmail.com). Nothing special needed.

Preconditions: Juice Shop running locally in the Kali VM, Burp Suite open to catch/replay requests. Also made a small PoC HTML page and served it with python3 -m http.server 8000, and tried it again as a plain local file to act like an attacker page.

Scope Limitations: Only looked at the username field on /profile. The change-password page was seen along the way but that's not part of this finding. Attacker page and victim page were both on 127.0.0.1, so it's not a true cross-site test yet — noted in 4.5.

3. Test Log

Date	Asset / Endpoint	Test case	Outcome	Evidence ID
27 Oct 2026	/profile	Baseline authenticated GET request	HTTP 200; profile page rendered with current username	E-01
27 Oct 2026	/profile (POST)	Submit username change via POST body in Burp Repeater	HTTP 302 Found; redirected to /profile, session cookie/token reissued	E-02
27 Oct 2026	/profile?username=... (GET)	Test whether endpoint also accepts username via GET query string	HTTP 200 OK; parameter accepted without rejection	E-03
27 Oct 2026	csrf-poc.html (python3 -m http.server 8000)	Host a crafted auto-submitting form targeting /profile	Page served; form submission logged against /profile	E-04
27 Oct 2026	/profile	Check victim profile after PoC execution	Username changed to "csrf_test_2" without direct in-app action	E-05
27 Oct 2026	csrf-username.html (file:// local page)	Repeat PoC as a local file to remove hosting dependency	Username changed to "csrf-lab-test" after opening local file	E-06
27 Oct 2026	/profile (browser DevTools Network)	Confirm request chain for PoC-driven change	POST /profile → 302 Found, followed by GET /profile → 200 OK	E-07

4. Finding Record (duplicate this section for each suspected/confirmed finding)

Finding ID / Title	F-02 – CSRF on /profile, can change someone's username without them doing it
Affected asset	http://127.0.0.1:3000/profile (localhost:3000/profile)
Method / account role	Logged in as a normal customer; attack run from a local HTML page while that session was active.
Severity / CVSS	Medium, roughly — state-changing action, no CSRF token. Not formally scored yet.

4.1 Summary

The username field on /profile can be changed by a request coming from another page, with no CSRF token and no second check. I made a small HTML page that auto-submits a form to http://127.0.0.1:3000/profile with a username value, ran it from a local python server and also just as a local file. While I was logged into Juice Shop in the browser, opening that page changed my username to "csrf_test_2" / "csrf-lab-test" without me doing it from the actual profile form. It also took the username as a plain GET parameter, so technically even a link or an image tag could trigger it, no form needed.

4.2 Preconditions

Victim just needs to be logged into Juice Shop and open the attacker's page/link while that session is still active. No password or anything else needed from the attacker side.

4.3 Reproduction Steps

1. Log in to Juice Shop, check current username on /profile.

2. In Burp Repeater, send POST /profile with username in the body — got 302 Found, username changed.

3. Tried GET /profile?username=<value> too — also worked, 200 OK.

4. Made csrf-poc.html with an auto-submit form pointing at /profile with a username value.

5. Served it with python3 -m http.server 8000, and separately just opened it as a local file (file:///home/rola/csrf-username.html).

6. With Juice Shop still logged in on the same browser, opened the poc page — request fired on its own, no prompt.

7. Went back to /profile — username had changed, even though I never touched the actual username form.

4.4 Expected vs. Actual Result

Expected: changing the username should need a CSRF token tied to the session, or at least check the Origin/Referer before accepting it.

Actual: it just accepted the request and changed the username, using whatever cookie the browser already had. No token, and GET worked just as well as POST.

4.5 Impact & Limitations

Someone could silently change a victim's username without them knowing, which isn't great for impersonation / could be chained with other issues. One limitation: I tested from the same machine (127.0.0.1 for both the victim session and the poc page), so some requests show Sec-Fetch-Site: same-origin — I'd want to redo this from an actual different origin to be fully sure SameSite cookies aren't quietly helping here. Didn't get password change or full account takeover out of this, just the username.

4.6 Remediation Recommendation

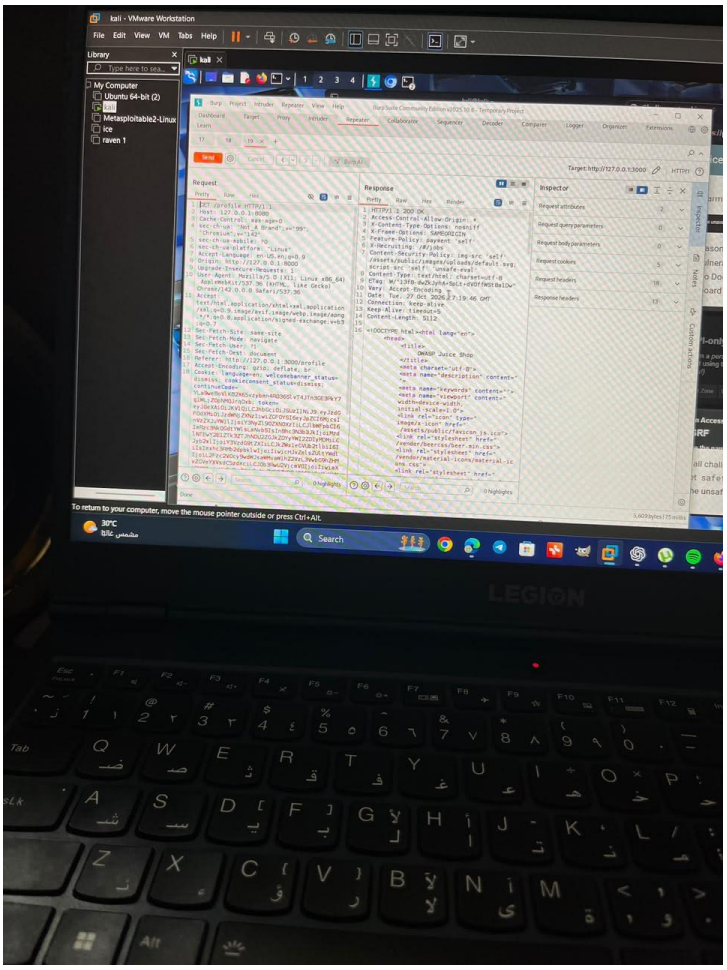
Add a proper CSRF token on /profile (and other state-changing endpoints) and check it server-side. Set cookies to SameSite=Strict/Lax. Check Origin/Referer on state-changing requests. Don't accept username changes via GET at all — POST/PUT only. Re-test from a separate origin once fixed.

5. Evidence Register

Filename / location	Related finding	Sensitive data redacted?
E-01 – /profile baseline response (Burp Repeater)	F-02 / context	Review before sharing
E-02 – POST /profile username=csrf_test_1, 302 Found	F-02	Session token visible – redact before sharing
E-03 – GET /profile?username=rrrrd, 200 OK	F-02	Review before sharing
E-04 – csrf-poc.html served via local HTTP server + access log	F-02	No sensitive data identified
E-05 – /profile showing username changed to csrf_test_2	F-02	Review before sharing
E-06 – Local file csrf-username.html PoC and resulting profile (csrf-lab-test)	F-02	Review before sharing
E-07 – Browser DevTools Network tab: POST /profile 302 → GET /profile 200	F-02	Review before sharing

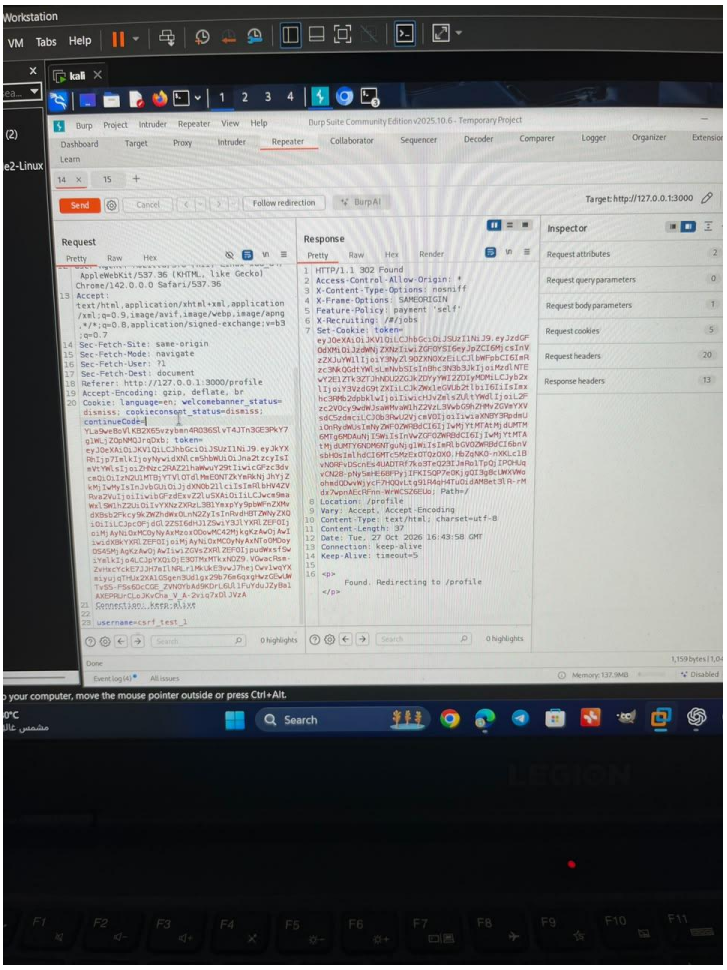
Appendix A – Screenshots

E-01 – /profile baseline (Burp Repeater)



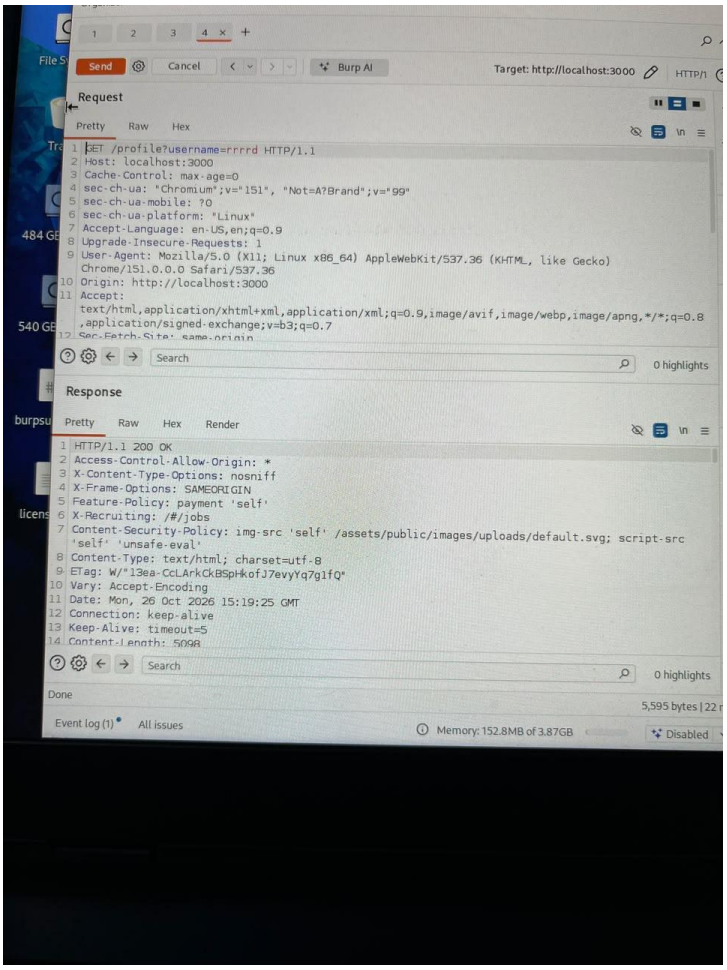
Baseline GET /profile through Burp Repeater, 200 OK.

E-02 – POST /profile, username=csrf_test_1



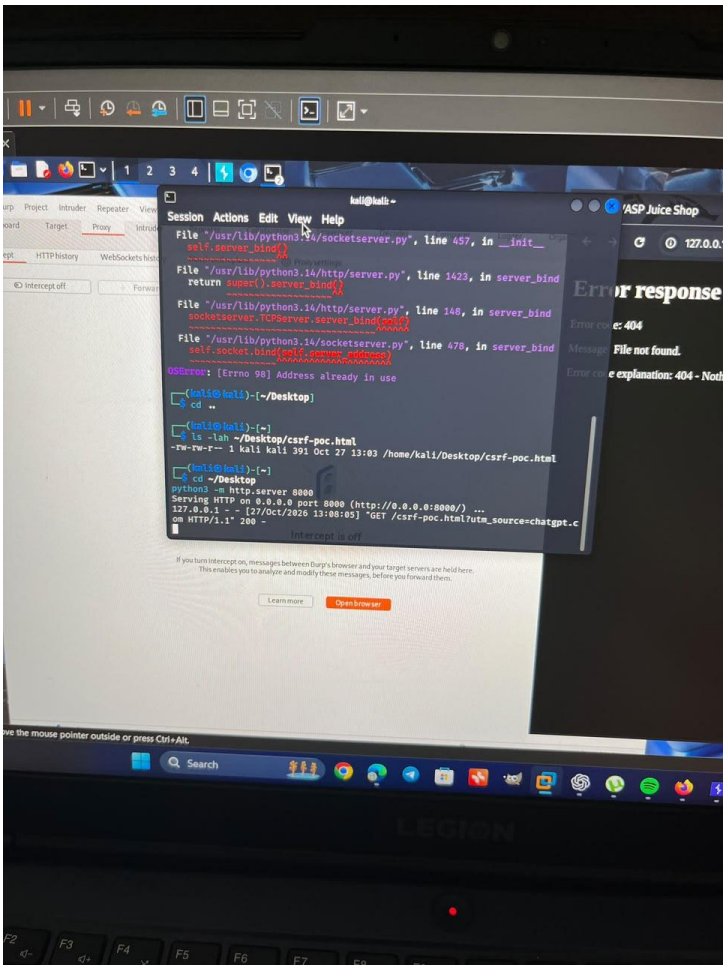
POST body with username field, 302 Found, Set-Cookie/token reissued.

E-03 – GET /profile?username=rrrrd

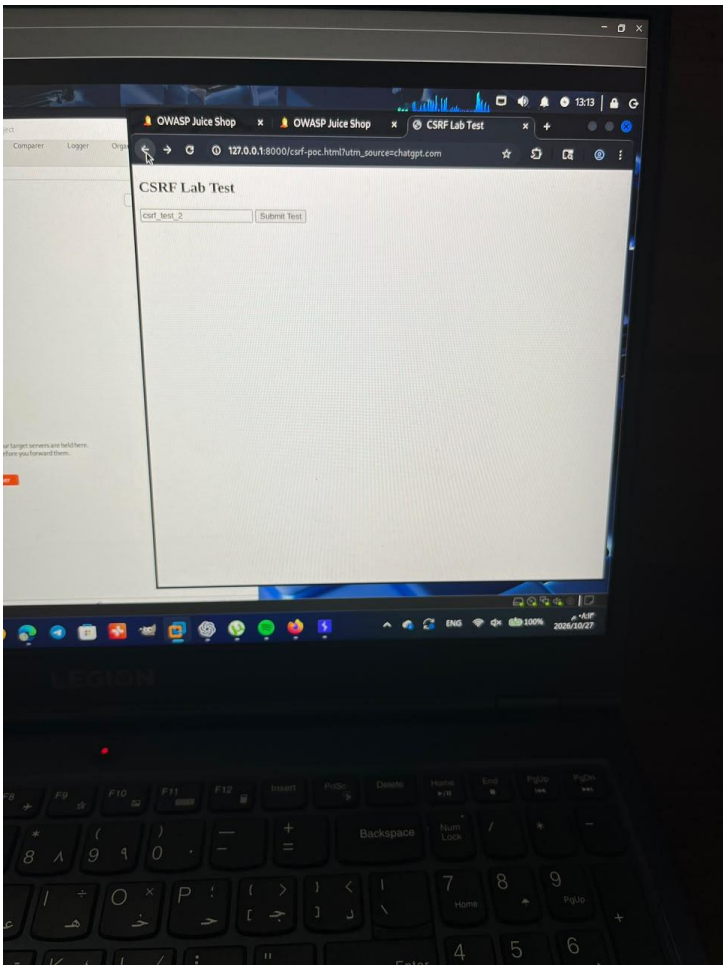


Same change accepted through a GET query string, 200 OK.

E-04 – csrf-poc.html served and opened

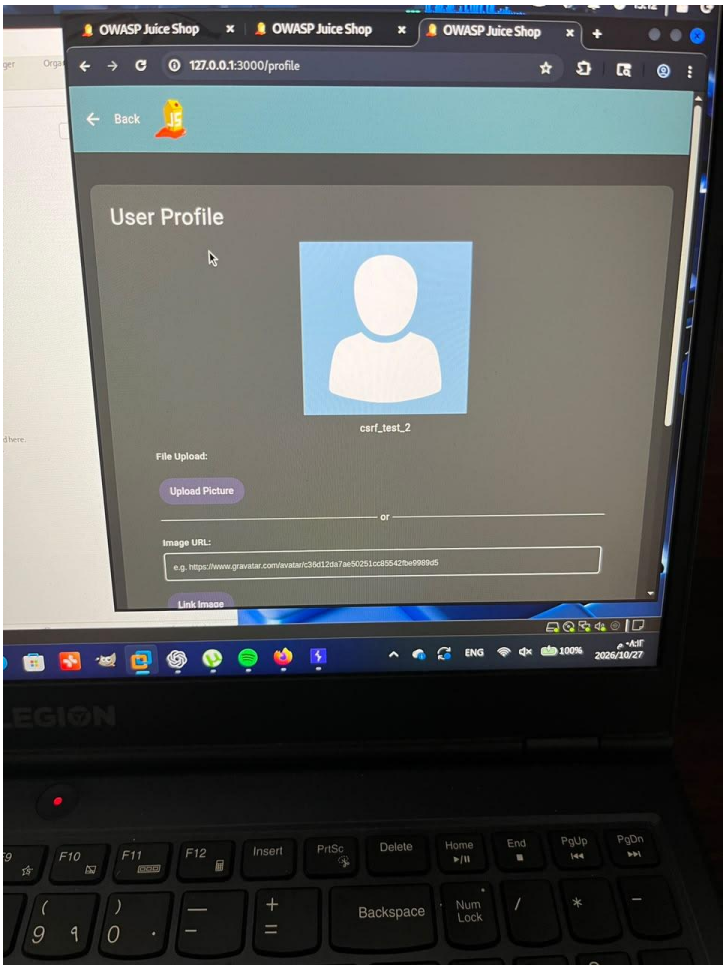


python3 -m http.server 8000 serving csrf-poc.html, request logged in the terminal.



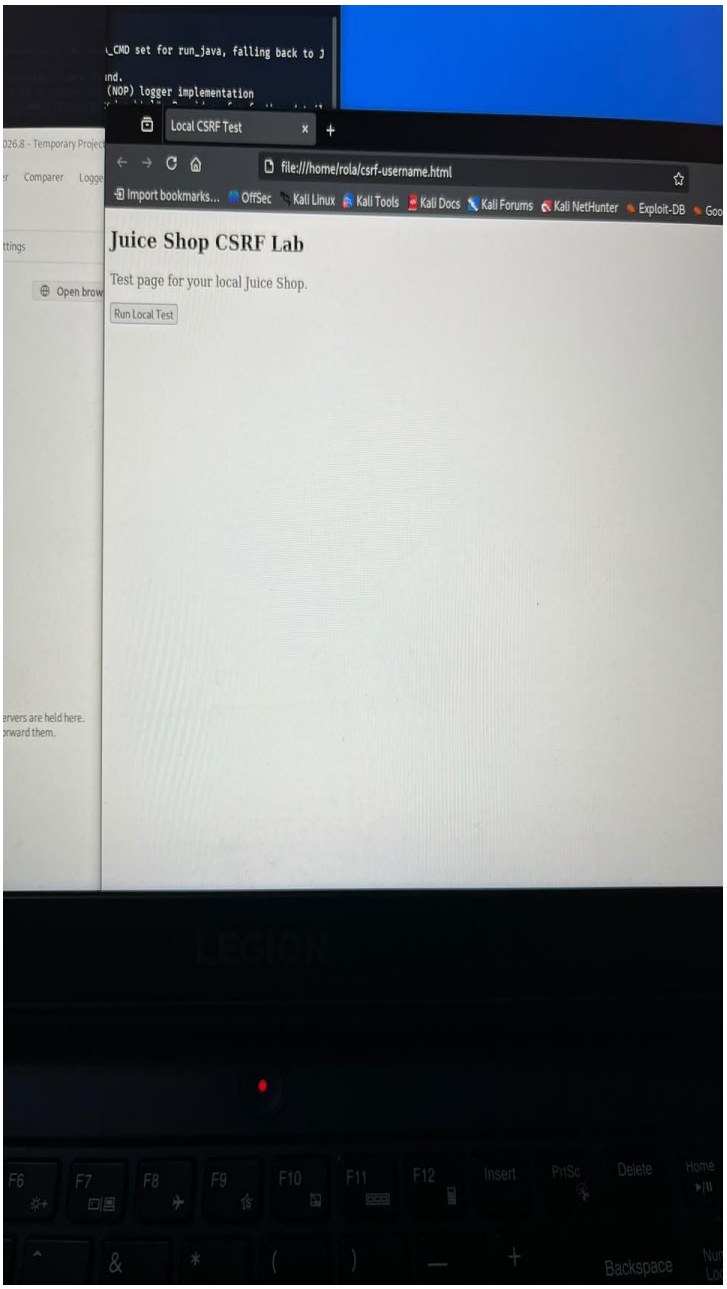
csrf-poc.html opened in the browser (CSRF Lab Test page).

E-05 – /profile after the PoC ran

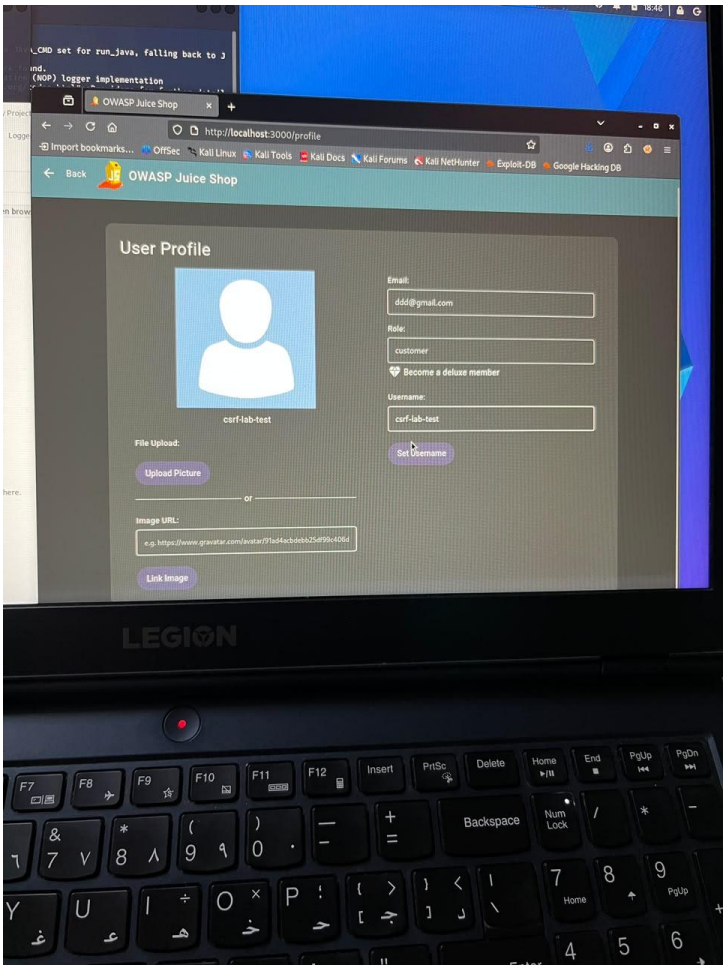


Username now shows csrf_test_2, changed without using the real profile form.

E-06 – local file version of the PoC

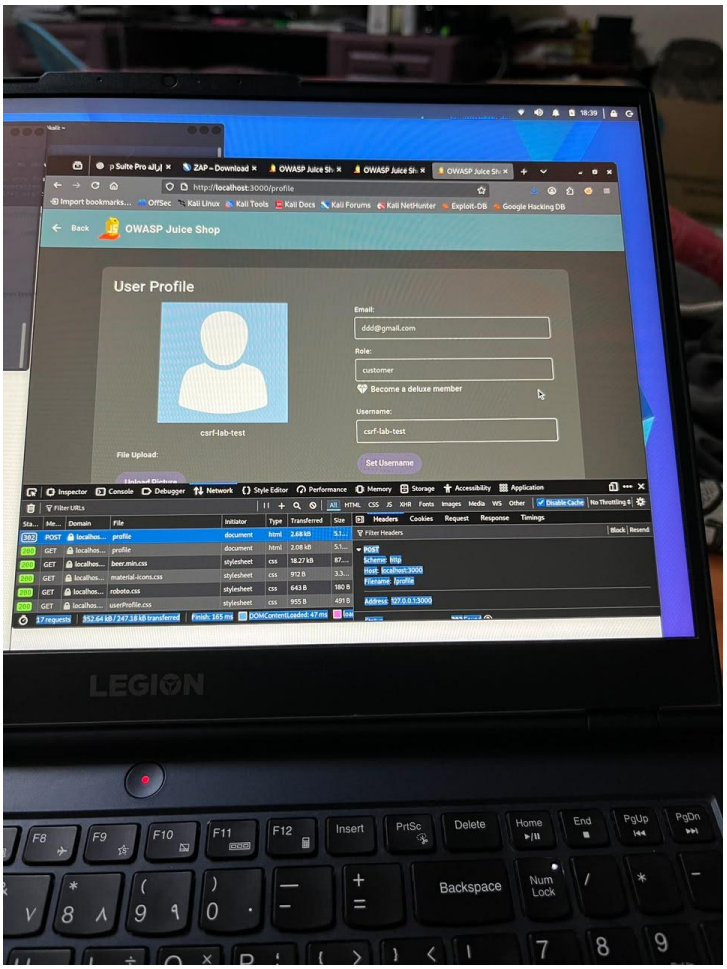


csrf-username.html opened directly as a local file (file:///home/rola/...).



Resulting /profile page, username changed to csrf-lab-test.

E-07 – DevTools network tab



POST /profile → 302, followed by GET /profile → 200, confirming the request chain.